User Guide: Employee Transfer

> Record, approve and apply dated employee moves — department, job position, work location, manager, company — as **new** `hr.version` history entries, without ever overwriting, duplicating or archiving the employee record.

- **Technical name:** `odomate_hr_transfer`
- **Version:** 19.0.1.0.4
- **License:** LGPL-3
- **Author:** OdoMate
- **Depends on:** `hr`, `mail`
- **Changelog (19.0.1.0.4):** fixed the demo "applied" transfer (`TRF/00001`), which showed a blank **From Department** because it was created directly in demo data with `state="applied"` — the compute that freezes that value never ran for it; the record now explicitly carries its pre-transfer department (Research & Development) so it reads "Research & Development → Customer Success" as intended.
- **Changelog (19.0.1.0.3):** fixed Ukrainian translations of Python messages (validation errors, confirmations, activity summaries) not loading on a `uk_UA` database — they were correctly written in `i18n/uk.po` but rejected by Odoo's loader; error messages now display in Ukrainian as expected.

Table of Contents

---

1. What does this module do

An employee moves from Sales to Customer Success on 1 October. In standard Odoo you would open the employee form and overwrite the department — and the old value is gone. This module replaces that with a small, auditable record: model `odomate.hr.transfer`, one record per move, with a reference like `TRF/00001`, an effective date, an approval step, and a chatter trail.

When the transfer is applied, the module **creates a new `hr.version` record** dated on the effective date. It does not edit the employee's existing version, does not create a second employee record, and does not archive the old one.

Capability	Details
Dated moves	Every transfer has an **Effective Date**. Applying it writes a `hr.version` whose `date_version` is that date.
Five dimensions	**To Department**, **To Job Position**, **To Work Location**, **To Manager**, **To Company**. Any combination; leave a field empty to mean "no change".
Before/after view	The form shows the current value (**From Department**, **From Job Position**, …) next to the requested one, so the change is readable at a glance.
Approval step	`draft → to_approve → approved → applied`, with `refused` and `cancelled` as alternate endings. Approval is restricted to HR Managers in code, not only in the UI.
Scheduled application	A daily scheduled action applies approved transfers whose effective date has arrived.
Audit trail	`mail.thread` chatter on every transfer: status changes, employee, effective date and each target field are tracked.
Immutability	Once applied, the employee, the effective date, the targets and the status can no longer be edited — attempting it raises a clear error. The "From …" values are frozen at that same moment, so the record permanently answers "moved from where" even after later changes to the employee.

2. Where to find it in Odoo

The module has no top-level app of its own. It attaches to the standard **Employees** app.

- **Employees ▸ Transfers** — the list of all transfer records (menu `menu_odomate_hr_transfer`, sequence 45, visible to `hr.group_hr_user`). The list shows Reference, Employee, Effective Date, From Department, To Department and Status, colour-coded: green for applied, blue for approved, grey for refused/cancelled.
- **Employee form ▸ Transfers smart button** — the button box on any employee form gets a **Transfers** stat button (exchange icon) showing the number of transfers for that employee. Clicking it opens the same list filtered on that employee, with the employee pre-filled on any

new record.

Ready-made filters in the search view: **Draft**, **To Approve**, **Approved**, **Applied**, **Refused**, **Cancelled**, and **Due for Application** (approved with an effective date on or before today). Group-by options: Employee, Status, Department, Company, Effective Date.

There is nothing to configure before first use. The sequence (`TRF/` + 5 digits) and the scheduled action are installed with the module.

---

3. Creating a transfer

Path: `Employees` ▸ `Transfers` ▸ **New** — or the **Transfers** smart button on the employee's form.

Field	Label on screen	Notes
`name`	Reference	Read-only, filled automatically from the `odomate.hr.transfer` sequence: `TRF/00001`, `TRF/00002`, …
`employee_id`	Employee	Required. Editable only while the transfer is in Draft.
`effective_date`	Effective Date	Required, defaults to today. Editable until the transfer is applied.
`new_department_id`	To Department	Optional. Empty = no change (the field shows the placeholder `No change`).
`new_job_id`	To Job Position	Optional.
`new_work_location_id`	To Work Location	Optional.
`new_parent_id`	To Manager	Optional. Another `hr.employee`.
`new_company_id`	To Company	Optional. Only shown when multi-company is enabled (`base.group_multi_company`).
`reason`	Reason	Free text, e.g. "Reorganisation of the post-sales teams".

The **What Changes** block is a two-column, before/after layout:

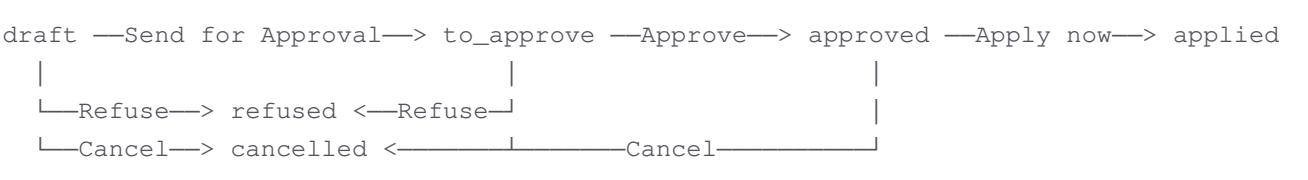
Left column ("Current")	Right column ("New")
From Department	To Department
From Job Position	To Job Position
From Work Location	To Work Location
From Manager	To Manager

From Company	To Company

The left column is always read-only. Before the transfer is applied it is computed live from the employee record, so it tracks any change made elsewhere. **Once the transfer is applied, the left column is frozen for good** — it keeps showing the values the employee actually had at that moment, even if the employee's department, job, work location, manager or company are changed again afterwards by a later transfer or a direct edit. This is what makes an applied transfer a reliable "moved from X to Y" record instead of a snapshot that quietly goes stale. **A row left empty on the right means "no change"** — that dimension is simply not touched when the transfer is applied.

Two more fields appear on the form only after the transfer is applied: **Applied On** (`applied_date`, the exact timestamp) and **Created Version** (`version_id`, a link to the `hr.version` record that was created).

4. The workflow: draft to applied



Which buttons appear in which state (they are hidden by `invisible=` conditions on the form header):

Status	Buttons in the header
Draft	**Send for Approval**, **Refuse**, **Cancel**
To Approve	**Approve** (HR Managers only), **Refuse**, **Cancel**
Approved	**Apply now** (only once the effective date has arrived), **Cancel**
Applied	none — the record is final and read-only
Refused	none
Cancelled	none

The status bar shows `Draft → To Approve → Approved → Applied`; `Refused` and `Cancelled` are terminal states outside that ribbon.

Send for Approval

Moves the transfer to **To Approve** and schedules a To Do activity (deadline = the effective date) for every HR Manager who has access to all companies involved (the employee's company and, for a cross-company move, the target company). If no such manager exists, the module logs a note in the server log and continues — sending for approval never fails because of a missing approver.

Approve

Only visible to members of `hr.group_hr_manager`, **and enforced inside `action_approve` itself**, not just by hiding the button. Two checks run:

1. The user must belong to `hr.group_hr_manager`, otherwise: "Only an HR Manager can approve employee transfer TRF/00007."
2. For a cross-company move, **To Company** must be one of the approver's allowed companies (`company_ids`), otherwise: "You cannot approve transfer TRF/00007: it moves the employee to Odoo Belgium, which is not one of your allowed companies."

Both are raised as `AccessError`, so calling the method from a script, an automation or the API is blocked exactly the same way as clicking the button.

On approval the pending To Do activities are closed with the feedback "Transfer approved."

Apply now

Apply now appears only when the status is `approved` **and** the effective date is today or earlier (the computed field `apply_ready`, labelled Effective Date Reached). While the transfer is approved but still in the future, the form shows a blue banner instead:

> Approved and waiting for the effective date. The **Apply now** button appears once that date has arrived, or the nightly scheduled action applies it automatically.

Clicking **Apply now** shows a confirmation dialog first:

> Applying is final: it writes a new dated version on the employee record and cannot be undone from this screen. Reversing this move means recording another transfer. Continue?

The same guard exists in `action_apply`: applying an unapproved transfer, or one whose effective date is still in the future, raises a `UserError`.

After applying, any attempt to change the employee, the effective date, the status or any of the five target fields raises:

> Transfer TRF/00007 has already been applied and can no longer be changed. Record a new transfer to reverse or correct this move.

The **Reason** text stays editable after applying, so a correction to the wording is still possible.

---

5. Validation rules

Two constraints are checked on every save (they run in Python via `@api.constrains`, so they apply to imports and API calls too).

Rule 1 — the transfer must change something

At least one of the five "To ..." fields must be set, and each one that is set must differ from the employee's current value.

- Nothing filled in → "Transfer TRF/00008 must change at least one of department, job position, work location, manager or company."
- **To Department** = the employee's current department → "To Department is already Customer Success for Marc Demo. A transfer must record an actual change."

The comparison is skipped for already-applied transfers — otherwise every historical record would become invalid the moment it took effect.

Rule 2 — the effective date must be after the last version

The effective date must be **strictly later** than `date_version` of the employee's most recent `hr.version` record.

Example: Marc Demo's latest version is dated **2026-06-11**. A transfer effective 2026-06-11 or 2026-05-01 is rejected:

> The effective date of transfer TRF/00008 must be later than 2026-06-11, the date of the most recent version of Marc Demo.

2026-06-12 or later is accepted. This keeps the version history strictly ordered and prevents two versions from sharing the same date. Practical consequence: **back-dating a transfer before the employee's most recent version is not possible** — if a move needs to be recorded retroactively, its effective date must still land after the last existing version.

---

6. What "Apply now" actually writes — worked example

The scenario

Marc Demo works in **Sales**, job position **Sales Manager**, work location **Building 1, Second Floor**, and his most recent `hr.version` is dated **2026-06-11**. HR moves him to the **Customer Success** department effective **2026-10-01**.

The record

Field	Value
Reference	`TRF/00007`
Employee	Marc Demo
Effective Date	2026-10-01
From Department	Sales
To Department	**Customer Success**
From Job Position	Sales Manager
To Job Position	(empty — no change)

From Work Location	Building 1, Second Floor
To Work Location	(empty — no change)
Reason	Reorganisation of the post-sales teams: moved to Customer Success to own the onboarding of new accounts.

The steps

1. **12 September 2026** — an HR Officer creates `TRF/00007` and clicks **Send for Approval**. Status → To Approve; the HR Managers get a To Do activity with deadline 2026-10-01.
2. **14 September 2026** — an HR Manager opens the record and clicks **Approve**. Status → Approved. The **Apply now** button is **not** shown yet: the effective date is still 17 days away, so the blue "waiting for the effective date" banner is displayed instead.
3. **1 October 2026** — either the HR Manager opens the record and clicks **Apply now**, or the nightly scheduled action gets there first. Status → Applied.

The version history afterwards

Marc Demo's employee record still has exactly one row in the employee list — nothing was duplicated or archived. His version history now reads:

date_version	department_id	job_id	work_location_id
2026-06-11	Sales	Sales Manager	Building 1, Second Floor
2026-10-01	**Customer Success**	Sales Manager	Building 1, Second Floor

The new version is built by copying the employee's current version and then overwriting only the dimensions the transfer actually changed — so wage, working schedule and every other versioned value carry over unchanged. Asking Odoo "which department was Marc Demo in on 15 August 2026?" correctly returns **Sales**; on 15 October 2026 it returns **Customer Success**.

On the transfer record itself, **Applied On** now shows `2026-10-01 02:00:14` and **Created Version** links straight to the new `hr.version`. The chatter gets a message:

> Transfer applied: a new version dated 2026-10-01 was added to the employee's history.

`TRF/00007` itself also keeps a permanent record of where Marc Demo moved **from**: **From Department** is frozen at **Sales** the moment the transfer is applied. If Marc Demo is moved again next year — by another transfer, or by editing `hr.version` directly — `TRF/00007` still reads "Sales → Customer Success"; it never starts showing the newer department on either side. Only a still-open transfer (draft, to approve, approved) keeps mirroring the employee live.

Manager and company are different

If the transfer had also set **To Manager** or **To Company**, those two would be written **directly onto the**

`hr.employee` **record** (`parent_id`, `company_id`) rather than into the version, because Odoo does not keep dated history for them. See Limitations.

7. Automatic application (scheduled action)

A scheduled action ships with the module:

Property	Value
Name	**Employee Transfer: Apply Due Transfers**
Technical id	`ir_cron_odomate_hr_transfer_apply`
Frequency	every **1 day**
Code	`model._cron_apply_due_transfers()`
Active	yes, from installation

Each run searches for transfers where status is `approved` **and** the effective date is on or before today, then applies each one through exactly the same `action_apply` method the button uses — the same status and effective-date guards apply, and the same `hr.version` is created.

Each transfer is applied inside its own database savepoint. If one fails — for example a validation error, or an access error on the target company — the failure is written to the server log ("Employee transfer TRF/00009 could not be applied automatically: ...") and the remaining transfers are still processed. A failed transfer simply stays in Approved and is retried the next night; it can also be applied manually with **Apply now**.

To watch it: `Settings` ▸ `Technical` ▸ `Automation` ▸ `Scheduled Actions` ▸ **Employee Transfer: Apply Due Transfers**. To disable automatic application entirely, untick **Active** there — transfers then only ever move to Applied when someone presses **Apply now**.

To see what the next run will pick up, open **Employees** ▸ **Transfers** and apply the **Due for Application** filter.

8. Refusing and cancelling

Both are non-destructive endings. **Neither writes anything at all to the employee record** — no version is created, no field on `hr.employee` is touched.

Refuse

Available in **Draft** and **To Approve**. Clicking **Refuse** opens a small dialog (the `odomate.hr.transfer.refuse` wizard) with one required field, **Refusal Reason**, and the placeholder "Explain why this transfer is refused. The employee record is left untouched."

Click **Refuse Transfer** to confirm, or **Discard** to back out. On confirmation the status becomes Refused, the reason is stored read-only in `refuse_reason`, and the pending approval activities are closed with the feedback "Transfer refused: ". The form then displays a red banner showing the

refusal reason at the top of the record.

Once refused, the transfer cannot be re-opened: "Transfer TRF/00008 can no longer be refused." Record a new transfer instead.

Cancel

Available in **Draft**, **To Approve** and **Approved** — including a transfer that was already approved but has not been applied yet. This is the way to stop a scheduled move before its effective date arrives: cancelling removes it from the scheduled action's queue.

Cancelling needs no reason, sets the status to Cancelled, and closes any pending approval activities with the feedback "Transfer cancelled." An applied transfer can no longer be cancelled: "Transfer TRF/00007 can no longer be cancelled."

---

9. Access roles and multi-company

Access is built entirely on the standard Employees groups — the module defines no groups of its own.

Role	Read	Create / Edit	Delete	Approve
Officer (`hr.group_hr_user`)	✓	✓	✗	✗
Administrator (`hr.group_hr_manager`)	✓	✓	✓	✓
Any other internal user	✗ (menu and smart button hidden)	✗	✗	✗

To assign: `Settings` ▸ `Users & Companies` ▸ `Users` ▸ select the user ▸ Human Resources section ▸ **Officer** or **Administrator**.

The same access rules apply to the refusal wizard `odomate.hr.transfer.refuse`, so any HR Officer can refuse a transfer they can see.

Multi-company

A global record rule, **Employee Transfer: multi-company**, restricts every read and write:

```
['|', ('employee_id.company_id', 'in', company_ids), ('new_company_id', 'in', company_ids)]
```

A transfer is visible when the employee's company **or** the target company is among the user's active companies. This deliberately keeps a cross-company move visible on both sides, so the receiving company's HR can see the incoming employee.

On top of that, `action_approve` refuses a cross-company transfer whose **To Company** is not in the approver's allowed companies — see section 4.

10. Demo data

If the database was created with demo data, the module installs three sample transfers plus two departments (**Customer Success** and **Field Operations**), one in each of the interesting states:

Reference	Employee	Effective date	Target	Status
auto	Marc Demo	90 days ago	Customer Success	**Applied** (with its `hr.version` linked)
auto	Anita Oliver	in 1 month	Field Operations	**Approved** — waiting for the effective date
auto	Barty McBly	in 14 days	Customer Success	**Draft** — not sent for approval yet

The approved one is the useful one to experiment with: open it and notice that **Apply now** is absent and the blue waiting banner is shown, because its effective date has not arrived.

11. Limitations

These are real gaps, stated plainly.

Topic	Limitation
Applying is final	There is no "un-apply", no reset-to-draft and no undo button. Once a transfer is applied its employee, effective date, status and targets are locked by a `UserError`. Reversing a move means recording **another transfer** back to the previous values (and its effective date must still be later than the version that was just created). Deleting the applied transfer is possible for an HR Administrator, but that does **not** delete the `hr.version` it created.
Manager and company are not dated history	Odoo does not version `parent_id` or `company_id` — they live only on `hr.employee`. A transfer that changes **To Manager** or **To Company** overwrites the current value on the employee record. This module therefore **cannot answer "who was this employee's manager on 15 August 2026?"** or "which company were they in last year". Only **department**, **job position** and **work location** get real dated history in `hr.version`.
Back-dating	The effective date must be strictly later than the employee's most recent `hr.version` date, so a move cannot be inserted between two existing versions or recorded retroactively before the last one.
No pay or contract changes	Wage, contract type, working schedule and any payroll field are carried over unchanged from the previous version. A promotion with a raise needs the salary handled separately in the employee's contract/version.

No bulk transfers	One record per employee. Re-organising a 20-person department means creating 20 transfers; there is no multi-select "transfer these employees" action and no import template beyond Odoo's generic CSV import.
No org-chart preview	The form shows the flat before/after field values only. There is no visual org chart, no preview of the resulting hierarchy, and no check that a **To Manager** change does not create a reporting loop.
No documents or reports	No transfer letter, no PDF report, no QWeb template, no email to the employee. The only notification is the internal To Do activity for HR Managers when a transfer is sent for approval.
No cross-company handshake	A cross-company move is a single approval by one manager who holds both companies. There is no two-sided "Send / Receive" flow, no acceptance step by the receiving company, and no inter-company document.
Single approval level	Exactly one approval step. There is no second approver, no delegation, no approval matrix by department or amount, and no escalation if the activity is ignored.
Scheduled action timing	The scheduled action runs once a day at whatever time the server schedules it. A transfer effective today may therefore only become Applied several hours into the day, unless someone presses **Apply now**.